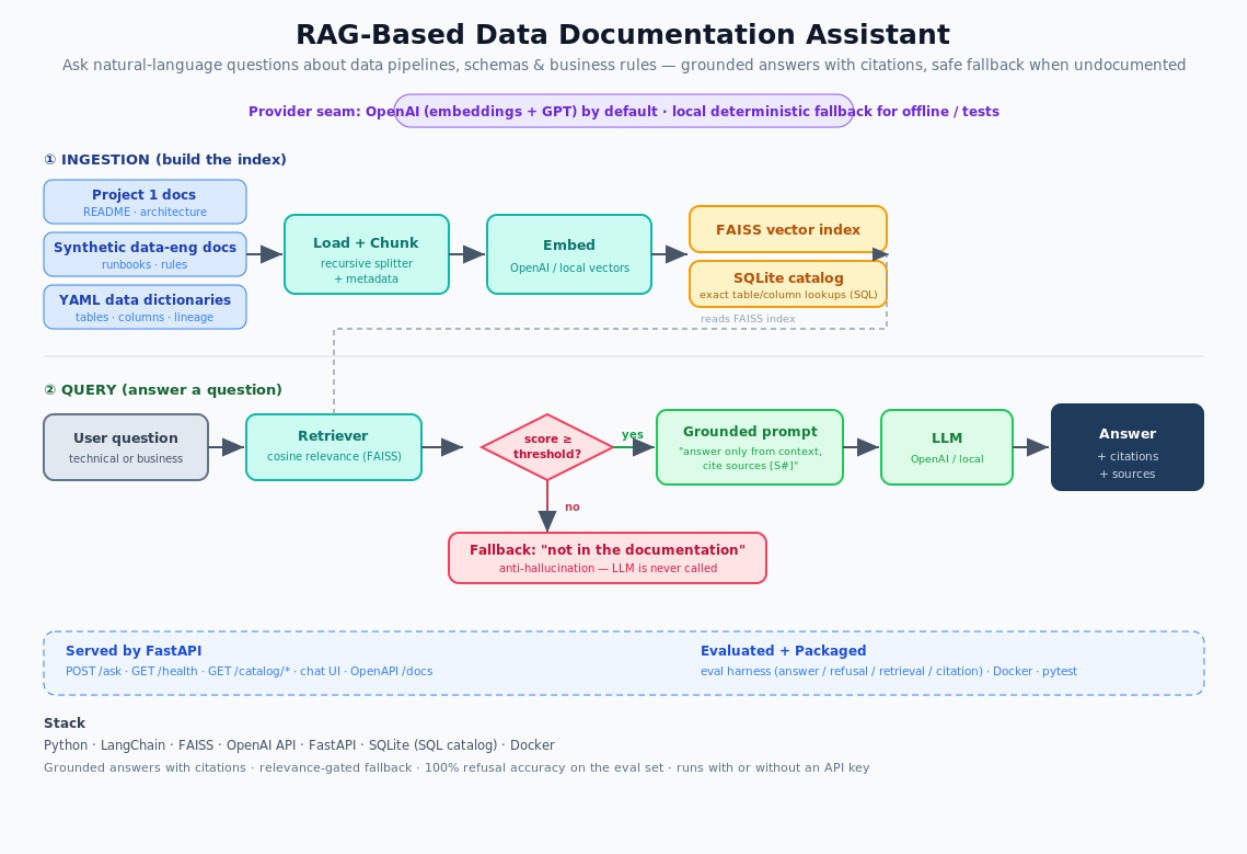
RAG-Based Data Documentation Assistant

Data / AI Engineering Portfolio — Project Report

A retrieval-augmented assistant that answers questions about data documentation with cited, grounded answers — and refuses to guess when the answer isn't documented.



Python · LangChain · FAISS · OpenAI API · FastAPI · SQLite · Docker

Author: Dhanush Battina

1. Executive summary

This project is a retrieval-augmented generation (RAG) assistant for data documentation. It ingests pipeline docs, schemas, data dictionaries and business rules, indexes them for semantic search, and answers natural-language questions with answers that are **grounded in the source documents and cited**. Critically, it **refuses to answer** when the documentation doesn't cover the question — the anti-hallucination behaviour that separates a trustworthy assistant from a demo.

It is OpenAI-powered, and engineered with a provider seam so the whole system — ingestion, retrieval, API and tests — also runs offline and deterministically without an API key. It intentionally consumes the documentation from Project 1 (the airline data platform), linking the two projects into one story.

2. What I built

- **Ingestion pipeline (LangChain):** load → chunk with metadata → embed → persist a FAISS vector index; plus a SQLite data catalog for exact lookups.
- **Grounded RAG chain:** relevance-scored retrieval, a context-only prompt with [S#] citations, and a threshold gate that returns a safe fallback without calling the LLM.
- **Provider seam:** OpenAI embeddings + GPT by default; a deterministic local provider (hashing embeddings + extractive LLM) when no key is present.
- **FastAPI service + chat UI:** /ask, /health, /catalog/* endpoints, OpenAPI docs, and a simple UI for technical and business users.
- **Evaluation harness:** answer accuracy, refusal accuracy, retrieval hit-rate and citation coverage on a labeled question set.

3. Architecture

Two phases. **Ingestion** turns documents into a normalized FAISS index and a SQL catalog. **Query** retrieves the most relevant chunks, gates them by a cosine relevance threshold, and — only if context clears the bar — asks the LLM to answer strictly from that context with citations. Below the bar, it returns the fallback and never calls the model.

4. Design decisions & trade-offs

Decision	Why
Refuse before calling the LLM	Stops hallucination at the source, not with post-hoc filtering.
OpenAI default + local fallback	Real quality with a key; still runnable and testable without one.
FAISS + SQLite together	Embeddings for meaning; SQL for exact schema/lineage facts.
Normalized index, cosine score	Interpretable [0,1] threshold that works across providers.
Evaluation harness	Treats answer quality + refusal as measurable, not vibes.

5. Evaluation results

On the labeled eval set with the local provider:

Metric	Result	Meaning
Answer accuracy	100%	answers the questions it should
Refusal accuracy	100%	refuses out-of-scope questions (no hallucination)
Citation coverage	100%	grounded answers carry a citation
Retrieval hit-rate	75%	expected source retrieved (higher with OpenAI)

6. What this project teaches (skills to speak to)

- How RAG works end to end: chunking, embeddings, vector search, grounded generation.
- Preventing hallucination with a retrieval-relevance gate and source-cited prompting.
- Combining semantic retrieval with a structured SQL catalog.
- Designing a provider abstraction so an LLM app is testable and reproducible.
- Serving an ML app behind a real API (FastAPI) and containerizing it.
- Evaluating an LLM system with objective metrics instead of anecdotes.

7. Interview talking points (Q & A)

Q. What is RAG and why use it here?

A. Retrieval-Augmented Generation grounds an LLM in your own documents: you retrieve the most relevant chunks and pass them as context so the model answers from facts rather than its parameters. It's ideal for documentation Q&A; because answers must be current, specific, and cite a source.

Q. How do you stop it from hallucinating?

A. Three layers: I retrieve with a cosine relevance score and drop anything below a threshold; if nothing clears the bar I return a fallback and never call the LLM; and when I do call it, the prompt instructs it to answer only from the provided context and cite [S#] tags. The eval set measures refusal accuracy explicitly.

Q. Walk me through retrieval.

A. Documents are chunked with metadata and embedded into a FAISS index that I L2-normalize, so Euclidean distance maps to cosine similarity. At query time I embed the question, take the top-k, and convert distance to a [0,1] relevance score for the threshold gate.

Q. Why FAISS and SQLite together?

A. FAISS answers fuzzy, semantic questions ('how are delays defined?'). SQLite answers exact structural ones ('what columns does fact_flights have?') that you shouldn't trust to embeddings. Pairing them covers both question types accurately.

Q. How do you evaluate a RAG system?

A. With a labeled set and four metrics: answer accuracy, refusal accuracy (the anti-hallucination signal), retrieval hit-rate (was the right source retrieved), and citation coverage. That turns 'it feels good' into numbers you can regress in CI.

Q. It's OpenAI-powered — how does it run without a key?

A. A provider seam. Embeddings and generation go through one interface with an OpenAI implementation and a deterministic local one (hashing embeddings + an extractive LLM). With no key it auto-selects local, so ingestion, the API and the whole test suite run offline; with a key it uses OpenAI for real quality.

Q. How would you improve retrieval quality?

A. Hybrid retrieval (BM25 + vector) plus a cross-encoder reranker, structure-aware chunking, and metadata filters per warehouse/doc-type. I'd also grow the eval set and track hit-rate as a regression metric.

Q. Cost and latency considerations?

A. Embeddings are computed once at ingestion; only the query embedding + one generation call happen per request. Caching, a smaller/cheaper chat model, and streaming keep cost and perceived latency down; the relevance gate also avoids paying for generation on out-of-scope questions.

8. How to run it

make setup → install · **make ingest** → build index + catalog · **make serve** → FastAPI + chat UI · **make eval** → metrics · **make test** → suite (no key needed). Set OPENAI_API_KEY in .env for real answers; otherwise the local provider is used automatically.

9. Putting it on your resume & LinkedIn

Resume bullet: Built a retrieval-augmented documentation assistant (Python, LangChain, FAISS, OpenAI, FastAPI, Docker) with source-cited, grounded answers, a relevance-gated anti-hallucination fallback, a SQL data catalog, and an evaluation harness (100% refusal accuracy on the eval set).

LinkedIn line: Shipped a grounded RAG assistant over data documentation — LangChain + FAISS retrieval, cited answers, an anti-hallucination gate, a FastAPI service, and objective evaluation. OpenAI-powered, runnable offline.